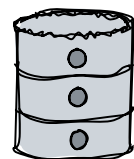
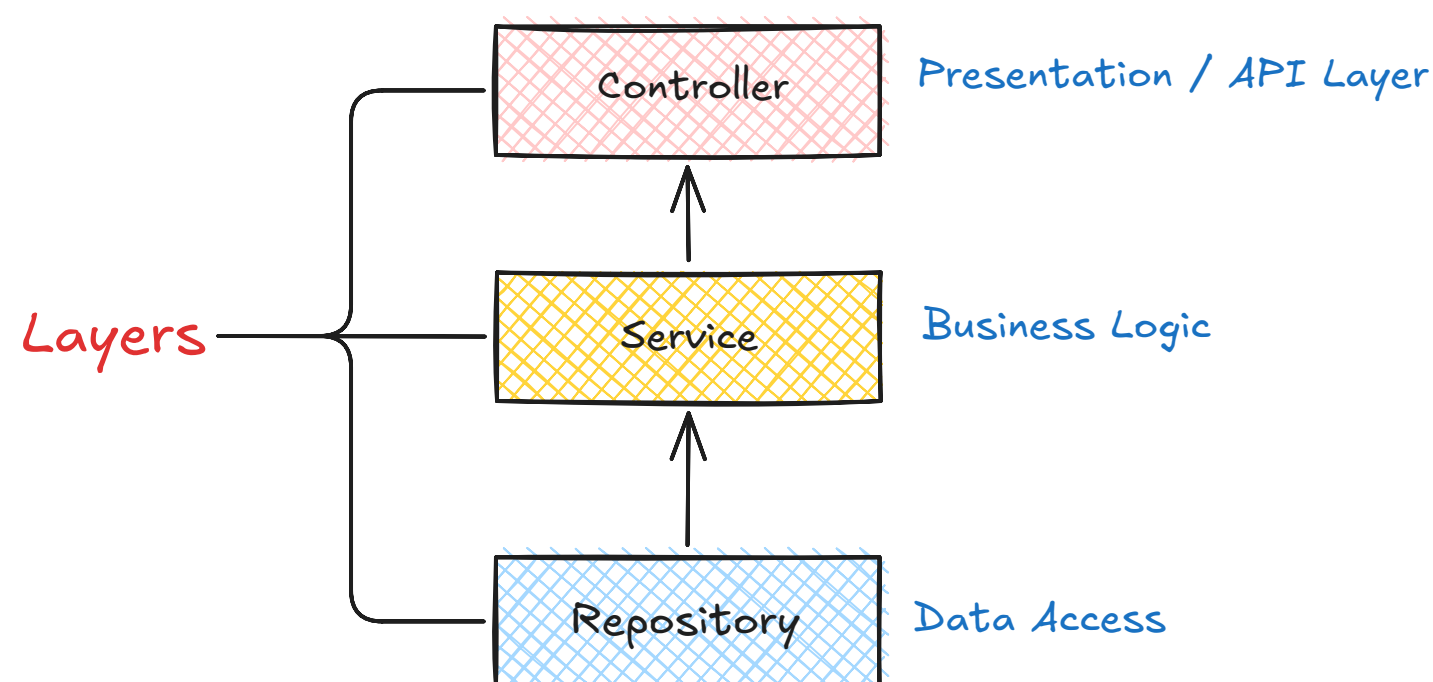


Validations and transformations



Backend Architecture



Repository Layer

Responsible for interacting with the database

Handles:

- Queries (SELECT, INSERT, DELETE, UPDATE)
- Database connection

Works with any storage:

- SQL (PostgreSQL, MySQL)
- NoSQL (MongoDB)
- Cache (Redis)



Service Layer

Contains the core logic of the application

Responsible for:

- Processing data
- Making decisions



- Calling one or more repository methods

- Triggering actions (emails, notifications, etc.)

The Service uses the Repository, It decides when and how to call it

```
call productRepository.getById()
call userRepository.getBalance()
```

Completely isolated from HTTP
Works with pure input/output data only

Controller Layer

This is the top layer of the architecture, and it is responsible for handling incoming API calls from clients.

Responsible for:

- Handling HTTP responses

Decides which status code to return:

200 → Success

400 → Bad Request

- Defining response format
- Validation & Transformation

API / Request Journey Through the Layers :

1. Starting Point: The Client

The client can be a browser, mobile app, or a tool like Insomnia/Postman.
Sends an HTTP request to the server

2. Routing (Route Matching)

When the request reaches the server, it goes through a Route Matching algorithm.

This determines which Controller method should handle this request.

3. Controller Layer (Validations & Transformations) Most critical step

The request enters the Controller, and before anything else happens, it passes through validation and transformation

4. Actual Work: Service Layer

Executes the business logic of the API:

- Send emails
- Perform calculations
- Prepare data for storage

5. Data Handling: Repository Layer

If the Service needs to read/write from the database:

- translates the request into database queries
- Executes queries on the database

6. Return Journey: Response

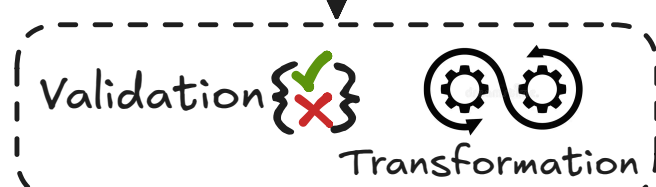
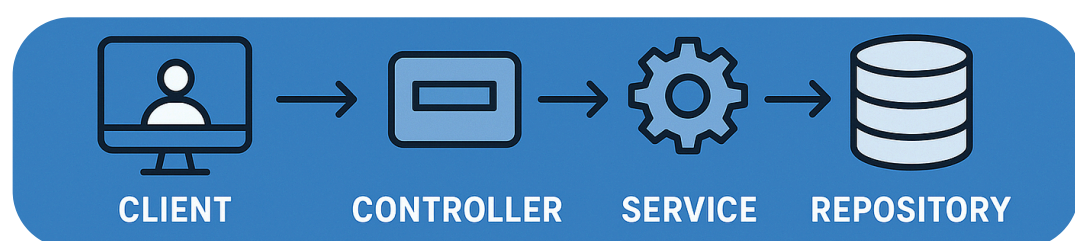
Service receives the result from the Repository

Completes any additional processing

Returns the final result to the Controller

Controller wraps the result in an HTTP response with a success code (e.g., 200 OK)

Response is sent back to the client



Validations and transformations

Server-side

1. Validations

Ensure that the data coming from the Client is correct, complete, and makes sense before it reaches the Service Layer.

a) Syntactic Validation

Checks the format/structure of the data.

Email: contains @ and a valid domain?

Phone number: all digits? Correct length?

JSON payload: fields exist and properly structured?

b) Semantic Validation

Checks the meaning/logical correctness of the data for your business logic.

User age ≥ 18 if it's adult content

Quantity ≤ stock available

End date > start date

c) Type Validation

to verify that the Data Type provided by the Client strictly matches the exact data type expected by the API

string "25" → number 25

string "true" → boolean true

2. Transformations

Pipeline

Prepare and clean the data so the Service Layer can use it safely.

Convert data types to what the Service expects:

"123" → 123 (String → Number)

"true" → true (String → Boolean)

Normalize text:

"EXAMPLE@EMAIL.COM" → "example@email.com" (lowercase)

" username " → "username" (trim whitespace)

Server-side validation is essential because client-side validation can be bypassed.